Project file structure

A practical guide to the files behind the facility inventory application.

Python / Flask | HTML, CSS, JavaScript | Supabase | Vercel



Vercel hosts the Python app and serves the frontend assets. Supabase holds the live records.

The main folders

```
bhc-inventory-management/
  app.py                    Application entry point
  inventory/                Main application code
    templates/              HTML page layouts
    static/                 CSS, JavaScript, icons
  database/                 SQL baseline, notes, database checks
  supabase/                 CLI configuration and migrations
    migrations/             Applied database changes, in order
  scripts/                  Setup, seed, build, verification tools
  tests/                    Automated Python tests
  public/static/            Generated assets for Vercel
  instance/                 Private local sessions and test data
  requirements*.txt         Python dependencies
  vercel.json               Hosting configuration
  .env.example              Local configuration template
  README.md                 Project overview
  GETTING_STARTED.md        Fresh-checkout setup instructions
  REQUIREMENTS.md            Product scope and rules
  DEPLOYMENT.md              Production setup and checks
```

Where to begin
Start with app.py, then inventory/__init__.py. Read templates/workspace.html and static/app.js together to understand how the interface is assembled.

Live app: bhc-inventory.vercel.app
Repository: ErrolKelvinTolentino/bhc-inventory-management

The Python backend

The backend receives browser requests, checks permissions and input, calls Supabase, and returns a page or updated data.

File	What it is for
`app.py`	Creates the Flask app and starts the local server. Vercel also loads the application from this entry point.
`inventory/__init__.py`	Configures Flask; defines workspace/API routes; handles inventory actions, request validation, errors, and local SQLite sessions.
`inventory/auth.py`	Handles account registration, login/logout, email-link callbacks, password recovery, and password changes.
`inventory/services.py`	Creates Supabase clients using the signed-in user and loads that user's facility, inventory, batches, and history.
`inventory/shared_sessions.py`	Stores encrypted server sessions in Supabase. Coordinates concurrent requests so login works across Vercel instances.
`inventory/demo.py`	Creates the isolated sample workspace and simulates stock operations for /demo. It does not seed the live database.

Example: receiving stock

1. The browser submits the receive-stock form.
2. Flask checks the session, request token, and selected facility.
3. The backend calls the authorized stock function in Supabase.
4. The database updates the batch and writes the movement history together.
5. Updated inventory data returns to the browser.

Sessions and inventory are different
A session remembers who is signed in. Locally, sessions normally use SQLite. On Vercel, sessions use encrypted Supabase storage. Live inventory always stays in Supabase.

The application currently supports facility administrators only. Each facility has its own workspace and records.

The frontend

HTML defines the page structure, CSS controls appearance, and JavaScript handles interactions and updates what users see.

inventory/templates/ - HTML layouts

File	What it is for
`workspace.html`	Main app shell: sidebar, navigation, content area, search, and dialogs.
`auth.html`	Login, registration, recovery requests, and facility-registration forms.
`callback.html`	Page used when returning from an email verification or password-reset link.
`password.html`	Password-change form for a signed-in or recovering account.
`error.html`	Fallback page when the workspace cannot load.

inventory/static/ - styles and browser behavior

File	What it is for
`app.css`	System-wide styling: typography, colors, spacing, tables, dialogs, and responsive layouts.
`app.js`	Renders dashboard/list content; controls search, filters, forms, dialogs, and inventory API calls.
`auth.js`	Controls account forms and submits authentication and onboarding requests.
`callback.js`	Passes email-link tokens to the backend for verification and clears them from the URL.
`password.js`	Submits password changes and displays validation or connection errors.
`icons.svg`	Reusable interface icons.
`favicon.svg`	Small icon shown in the browser tab.

One shared workspace layout
Dashboard, medicines, supplies, and history use workspace.html. Their individual content is currently generated by app.js, rather than separate HTML files for every screen.

Database structure and changes

SQL files describe the database structure and rules. The actual live accounts, inventory, and history are stored remotely in Supabase.

Where the database files live

File	What it is for
`database/supabase_setup.sql`	Original one-time baseline: tables, constraints, policies, triggers, views, and stock functions.
`supabase/config.toml`	Configuration used by the Supabase CLI.
`supabase/migrations/`	Deployable, ordered database changes. This is the migration folder used by the CLI.
`database/migrations/`	Planning notes only. Its README points to the actual Supabase migration folder.
`database/tests/`	SQL deployment checks plus verify.mjs, an isolated PostgreSQL test harness for the original schema.

The three current migrations

File	What it is for
`20261019000100_initial_inventory.sql`	Creates the initial facility inventory schema, access policies, and stock/history functions.
`20261021000100_administrators_only.sql`	Removes staff memberships and restricts the membership role to administrator.
`20261021000200_shared_sessions.sql`	Adds private encrypted session storage and server-only session coordination functions.

What the main records represent

Identity and access: Supabase Auth users, profiles, facilities, and facility_memberships.
Inventory: categories, items, and inventory_batches.
History: stock_transactions, stock_transaction_lines, and audit_events.
Server sessions: app_server_sessions.

A fresh checkout does not need a database reset
The shared project is already migrated. Future schema changes belong in a new migration. The baseline file remains a record of the original setup.

Maintenance tools and tests

The scripts folder contains commands for setup, deployment, sample data, and verification. These are not the website's normal page handlers.

scripts/

File	What it is for
`seed_supabase.py`	Creates/reuses the sample administrator, categories, items, and stock history. Reuses operation keys to avoid duplicate seed movements.
`configure_auth.py`	Configures allowed email callback URLs and optionally the Supabase Auth Site URL.
`configure_vercel.py`	Adds missing production environment variables to the linked Vercel project without printing secret values.
`build_vercel.py`	Copies inventory/static assets into public/static during the Vercel build.
`verify_live.py`	Exercises the Flask/Supabase integration, including stock workflows, account flows, facility isolation, and shared sessions.
`verify_deployed.py`	Checks the hosted website: authentication, seed inventory/history, page access, secure cookies, isolation, and assets.
`remove_seed_staff.py`	Historical cleanup tool used to remove the former sample staff login while retaining movement attribution.

tests/ - automated Python checks

File	What it is for
`test_app.py`	Inventory operations, demo isolation, request protection, history, expiry, and administrator-only behavior.
`test_auth.py`	Account flows, session rotation, logout invalidation, email callbacks, and password validation.
`test_deployment.py`	Serverless configuration, encrypted session behavior, secure cookies, and exclusion of local credentials.

Offline tests versus live checks
The tests/ suite runs without modifying the hosted inventory. Live verification scripts use labeled sample accounts and real Supabase connections; they are maintenance tools, not ordinary onboarding commands.

Configuration and local files

Configuration tells the same application how to run on a developer's computer or on Vercel.

File	What it is for
`.env / .env.example`	Your local settings and the shareable template. The real .env is ignored by Git.
`.env.local`	Local environment file generated by Vercel tooling; also ignored by Git.
`requirements.txt`	Python packages needed to run the application.
`requirements-dev.txt`	Runtime dependencies plus development tools such as pytest.
`requirements-lock.txt`	Exact package versions recorded from the development environment.
`vercel.json`	Flask preset, build command, function limit, bundle exclusions, and static-asset headers.
`.python-version`	Specifies Python 3.12 for the project.
`.gitignore / .vercelignore`	Define what Git and Vercel should exclude from source tracking or deployment uploads.

Documentation

README.md: overview and commands. **GETTING_STARTED.md:** fresh-checkout setup.
REQUIREMENTS.md: features and rules. **DEPLOYMENT.md:** production configuration and checks.

Generated and local folders

File	What it is for
`.venv/`	Installed Python interpreter environment and packages for this checkout.
`instance/`	Local sessions, signing key, generated sample credentials, and temporary test data.
`public/static/`	Generated copies for Vercel. Edit the originals in inventory/static/.
`artifacts/`	Screenshots and verification outputs.
`.vercel/`	Local project-link metadata for Vercel tooling.
`.git/`	Repository history and Git metadata.
`__pycache__/` `.pytest_cache/`	Generated Python bytecode and pytest cache files.

First edits: appearance in app.css; interface behavior in app.js; account behavior in auth.py; server inventory actions in __init__.py; database rules through new SQL migrations.